

DAY 04: JAVA CONCURRENCY I

Data Bytes Technologies · Java 25 & Modern Stack

Concurrency has a scary reputation. Today we take the fear out of half of it: what a thread really is, why we let the JVM manage them, and why Java 25 lets you launch **a million** of them. We end by *watching* the one thing that makes concurrency genuinely hard, which is exactly what Day 5 fixes.

// WHERE WE ARE: DAY 4 OF 10

You have the toolbox and the language. Today we make the language do *many things at once*.

Behind us

- Day 1: Git
- Day 2: Docker & Compose
- Day 3: **Java for polyglots**: records, sealed types, pattern matching, streams

This week's concurrency arc

- **Day 4: Concurrency I** (you are here): threads + virtual threads
- Day 5: Concurrency II: making shared state *safe*

💡 Yesterday's **records** were small, immutable values. That immutability is about to matter a lot: the trouble in concurrency is always **shared *mutable* state**, and records sidestep it.

// BY THE END OF TODAY, YOU CAN...

- explain what a **thread** really is: an OS thread, its stack, and the scheduler that runs it
- tell **concurrency** apart from **parallelism**, and know which one you're buying
- run work on an `ExecutorService` instead of hand-managing `Thread` objects
- explain **why virtual threads** make blocking cheap, and spin up **one per task**
- spot the **pinning** trap, and know when virtual threads help (I/O) and when they don't (CPU)
- *see a **race condition** happen*: the cliffhanger Day 5 resolves

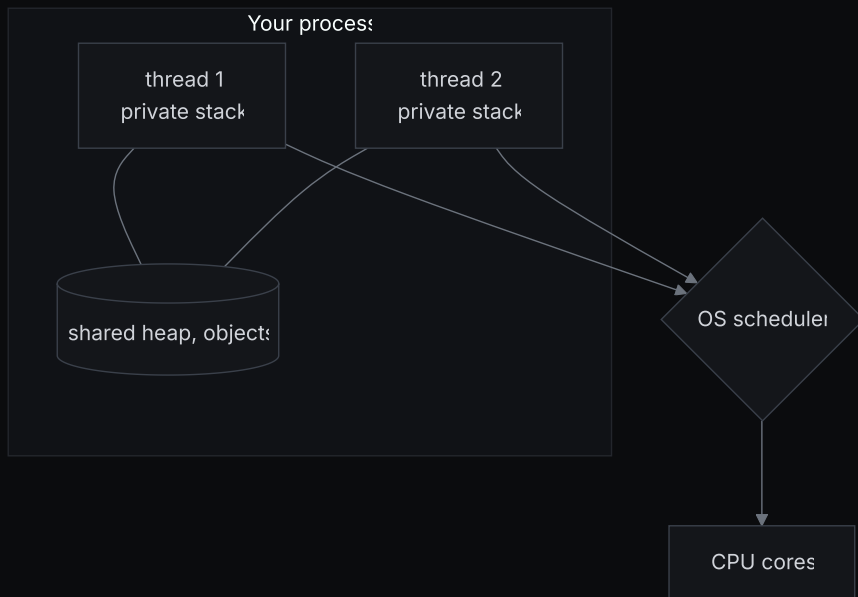
// FIRST, IN PLAIN TERMS: WHAT IS A THREAD?

You write code as a list of steps. A **thread** is one worker reading that list, top to bottom. To do two things at once (serve two requests, read two files) you need two workers reading two lists.

💡 In plain terms: a **platform thread** (the only kind Java had until recently) is a thin wrapper over an **operating-system thread**. It has its own **stack** (its private scratch space), and the **OS scheduler** decides when it gets a slice of a CPU core.

- OS threads are **heavy**: each reserves roughly **1 MB** of stack up front.
- The OS schedules them, and **switching** between them costs real time.
- A few **thousand** is a practical ceiling on a normal machine.

// ONE PICTURE: PROCESS, THREADS, STACKS, SCHEDULER



// CONCURRENCY IS NOT PARALLELISM

People use these as synonyms. They're not, and the difference decides which tool you reach for.

💡 **In plain terms:** **Concurrency** is *dealing with* many things at once (structure: many tasks in flight, taking turns). **Parallelism** is *doing* many things at once (execution: many cores literally running at the same instant).

- One barista taking ten orders and juggling them = **concurrency** (one core, tasks interleaved).
- Ten baristas each making one drink = **parallelism** (ten cores, truly simultaneous).
- Most server work is **concurrency over waiting**: 10,000 requests, each mostly *waiting* on a database. You don't need 10,000 cores. You need 10,000 cheap things that can wait.

// THE CLASSIC WAY: A `Thread` RUNS A `Runnable`

A `Runnable` is just "a chunk of work": one method, `run()`. A `Thread` runs it on its own.

📄 **Code along:** type this with me in `jshell` after the next slide.

```
Runnable work = () -> System.out.println(
    Thread.currentThread() + " is working");

Thread t = new Thread(work);
t.start();           // runs work() on a NEW thread, returns immediately
t.join();            // wait here until that thread finishes

// You almost never write this by hand in real code. The next slides show why.
```

`start()` is the magic word: it does **not** call `run()` on *your* thread. It hands the work to a new one and returns instantly. (Call `run()` directly and it just runs inline, a classic beginner trap.)

// TRY IT NOW (3 MIN): WATCH THREADS INTERLEAVE

Launch `jshell` and run two threads that print as they go:

```
jshell> Runnable count = () -> {  
    ...>     for (int i = 0; i < 5; i++)  
    ...>         System.out.println(Thread.currentThread().getName() + ": " + i);  
    ...> };  
jshell> var a = new Thread(count, "A");  
jshell> var b = new Thread(count, "B");  
jshell> a.start(); b.start();
```

The lines from **A** and **B** come out **interleaved**, and the order changes each run. Nobody chose that order. The **scheduler** did. That loss of control is the heart of concurrency.

// WHY WE DON'T HAND-MANAGE THREADS

`new Thread()` works, but on real systems it scales badly and leaks responsibility.

- **Creation is expensive:** a fresh OS thread per task means ~1 MB and a syscall every time.
- **No reuse:** you spin one up, use it once, throw it away. Wasteful under load.
- **No back-pressure:** 10,000 requests → 10,000 threads → out of memory. Nothing caps it.
- **No results, no error plumbing:** `Runnable` returns nothing and swallows exceptions.

We want to say *"here is some work, run it for me, give me the result"*, and let something else own the threads. That something is an **executor**.

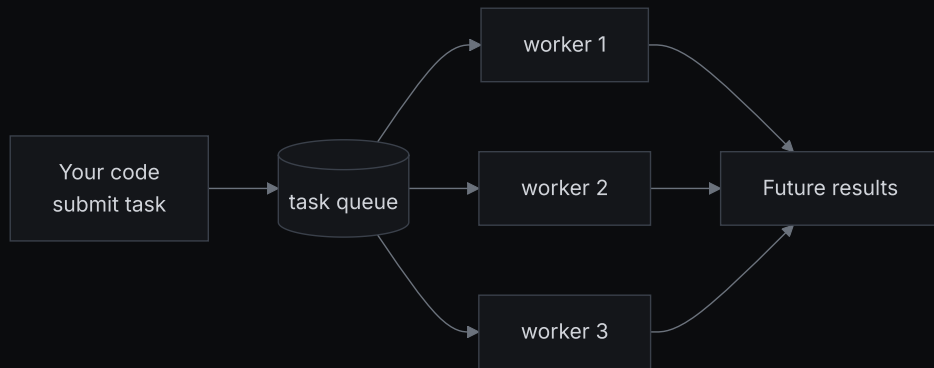
// EXECUTORSERVICE : SUBMIT TASKS, NOT THREADS

You hand the executor **tasks**; it owns the threads and hands you a **Future** for each result.

```
try (var pool = Executors.newFixedThreadPool(4)) {    // 4 reusable worker threads
    Future<Integer> f = pool.submit(() -> countRows(file));    // a Callable: returns a value
    Integer rows = f.get();    // block here until the task is done, then read the result
    System.out.println(rows + " rows");
}    // try-with-resources closes the pool: it waits for all tasks, then stops the threads
```

- A **Callable<T>** is a **Runnable** that **returns a value** (and may throw).
- **submit** returns a **Future<T>**, a handle to a result that *isn't ready yet*.
- The **try**-with-resources block **waits for every task** before closing. No leaked threads.

// ONE PICTURE: TASKS FLOW THROUGH A POOL



Takeaway: the **pool size** is the dial for *how many tasks run at once*. The threads are reused, created once, not per task.

// THE CATCH WITH POOLS: HOW BIG?

A fixed pool caps concurrency, which is good for CPU work and **bad** for waiting work.

- Pool of 4, and all 4 threads are **blocked** waiting on the database? The 5th task **waits in the queue**, even though the machine is doing nothing.
- Size it too **small** → tasks starve waiting their turn. Too **big** → you're back to thousands of expensive OS threads and you run out of memory.
- For I/O-heavy work there's no good number: you'd need one thread per in-flight request, and OS threads are too expensive to have that many.

So the real question becomes: *what if a thread that's just **waiting** were almost free?* Then you'd never have to pick a pool size for I/O at all.

// THE REAL VILLAIN: BLOCKING WASTES A PRECIOUS THREAD

When a thread calls the database or a remote API, it **blocks**: it sits idle, holding its ~1 MB OS thread, doing nothing but waiting for bytes to come back.

💡 **In plain terms:** a blocked platform thread is an **expensive resource doing nothing**. With a few thousand of them, your server spends most of its memory on threads that are just *waiting*.

Real services are mostly *waiting*: a typical request touches a DB and two other services and spends **95%+ of its life blocked**. Under the old model, that's 95% of your precious threads, wasted.

// THE OLD ESCAPE, AND WHY IT HURT

To stop wasting threads, the industry went **asynchronous**: don't block; register a **callback** for when the data arrives, and free the thread meanwhile.

- It scales, but it **colours your whole codebase**: once one method is async, every caller must be too, all the way up.
- A stack trace tells you little; `try/catch` and loops get awkward; the simple top-to-bottom read of your code is gone.
- You were forced to choose: **simple but slow** (blocking) or **fast but tangled** (async).

Virtual threads **dissolve that choice**: keep the simple blocking style *and* the scale.

// VIRTUAL THREADS, IN PLAIN TERMS

💡 **In plain terms:** a **virtual thread** is a thread the **JVM** manages, not the OS. It costs about as much as an ordinary object, so you can have **millions**. When one **blocks** on I/O, the JVM quietly sets it aside and runs another on the same OS thread, so blocking is cheap again.

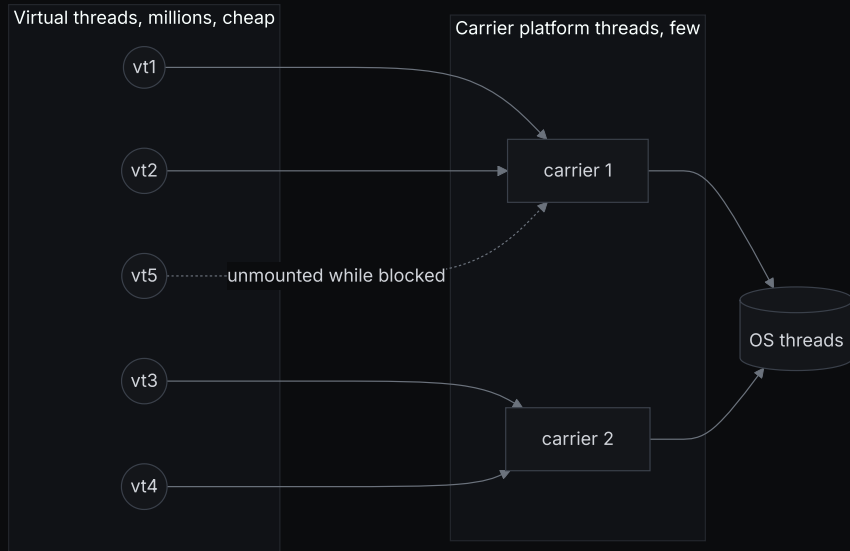
- Keep writing plain, **blocking, one-task-per-request** code: no callbacks, no colouring.
- The expensive layer (OS threads) stays small and busy; the cheap layer (virtual threads) grows as large as your workload needs.
- **Final in Java 25:** production-ready, no flags. Reach for it by default.

// PLATFORM VS VIRTUAL: THE NUMBERS

	PLATFORM THREAD	VIRTUAL THREAD
Managed by	the OS	the JVM
Stack cost	~ 1 MB reserved	a few hundred bytes , grows on demand
How many?	a few thousand	millions
Blocking on I/O	wastes the OS thread	cheap : it unmounts
Best for	CPU-bound work	I/O-bound work (waiting)

Same `Thread` API, same blocking calls, same `try/catch`. The only thing that changed is the **price of having one**, and that changes everything about how you design a server.

// ONE PICTURE: VIRTUAL THREADS RIDE ON CARRIERS



The dashed arrow is the whole trick: when `vt5` blocks, it **unmounts** from its carrier so the carrier can run another virtual thread. The OS thread is never left idle.

// THE IDIOM: ONE EXECUTOR, ONE TASK EACH

Never pool virtual threads. That re-imposes the old ceiling. Use the **per-task** executor: it mints a fresh virtual thread for **every** job you submit.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<Integer>> totals = files.stream()
        .map(file -> executor.submit(() -> countRows(file))) // one vthread per file
        .toList();
    int grandTotal = 0;
    for (var future : totals) grandTotal += future.get(); // collect results
} // close waits for every task to finish
```

A virtual thread per file, **all blocking on disk at once**, and the code reads top-to-bottom like a single thread. The long method name is the point: **per task**, not pooled.

// TRY IT NOW (4 MIN): A *MILLION* VIRTUAL THREADS

In `jshell`, prove that millions of blocking threads are cheap:

```
jshell> var done = new java.util.concurrent.atomic.AtomicLong();
jshell> var start = System.currentTimeMillis();
jshell> try (var ex = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
...>     for (int i = 0; i < 1_000_000; i++)
...>         ex.submit(() -> { Thread.sleep(100); done.incrementAndGet(); return null; });
...> }
jshell> done.get()
jshell> System.out.println((System.currentTimeMillis() - start) + " ms")
```

A **million** threads, each *blocking* (sleeping) 100 ms, and your laptop shrugs. Try that with platform threads and you'd hit `OutOfMemoryError` in the low thousands.

// 🖐️ EVERYONE WITH ME?

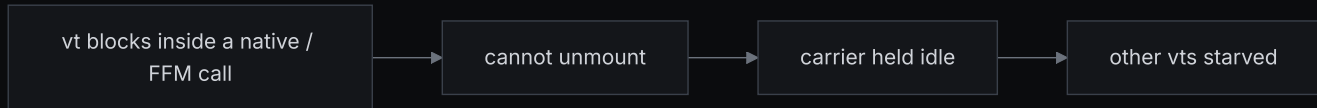
You should, right now, be able to say:

- a **platform thread** is an OS thread (~1 MB); a **virtual thread** is JVM-managed and ~free
- blocking is **cheap** on a virtual thread because it **unmounts** from its carrier
- the idiom is `newVirtualThreadPerTaskExecutor()`: one task each, never a fixed pool
- virtual threads are **final in Java 25**: safe to ship, no flags

If any of those is fuzzy, flag it now: the pinning trap and the race demo both build on this model.

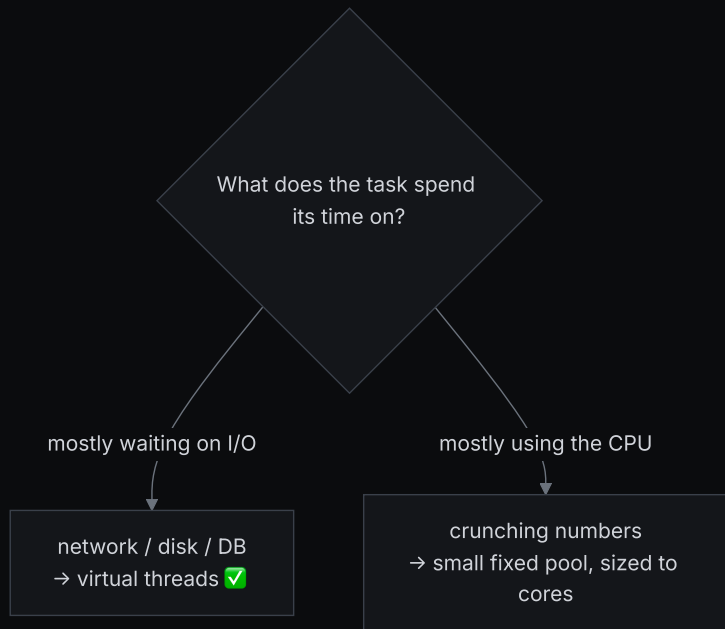
// THE ONE TRAP: PINNING (AND WHAT JDK 25 ALREADY FIXED)

A virtual thread's superpower is **unmounting** when it blocks. **Pinning** is when it *can't*: stuck on its carrier, holding an expensive OS thread hostage.



💡 **In plain terms:** through JDK 23, blocking inside a `synchronized` block pinned the thread. **JDK 24's JEP 491 fixed that:** `synchronized` no longer pins. In **JDK 25** the remaining cause is blocking inside **native code** (JNI / the Foreign Function & Memory API).

// WHEN DO VIRTUAL THREADS ACTUALLY HELP?



💡 **In plain terms:** virtual threads win when threads spend their time **waiting**. For pure **CPU-bound** crunching they add nothing: you only have so many cores, so a small **fixed pool** is right. A million virtual threads doing

// NOW THE CATCH: SHARED MUTABLE STATE

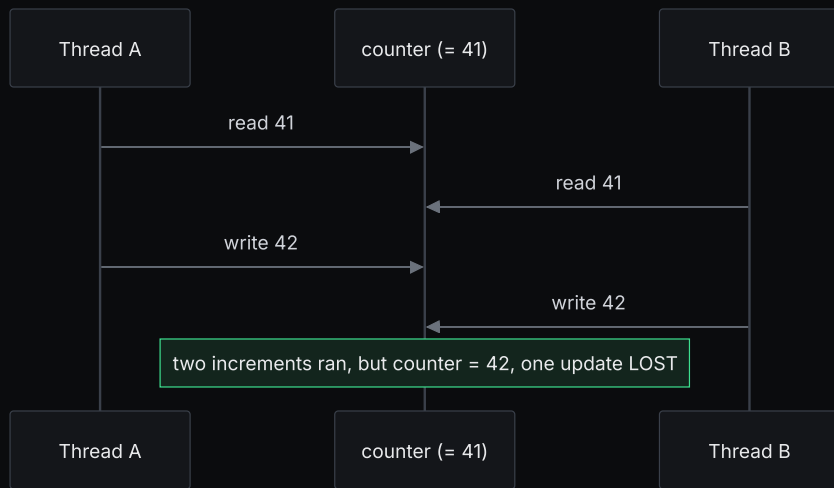
So far, easy: cheap threads, simple blocking code. Here's the part that's genuinely hard, and it has **nothing to do** with virtual vs platform threads.

- Threads each have a **private stack**, but they **share the heap**, the same objects.
- The moment **two threads write the same mutable object**, the scheduler's freedom to pause either one *between any two steps* becomes a problem.
- Remember the interleaving from this morning? That same unpredictability, applied to **data**, silently corrupts it.

Let's make it concrete with the simplest possible operation: adding one to a counter.

// ONE PICTURE: A RACE CONDITION, CONCRETELY

Two threads each do `counter++` on a shared counter that starts at **41**.



Both read **41** before either wrote. Two increments happened; the counter advanced by **one**. That silently-wrong result is a **race condition**, and Day 5 is largely about preventing it.

// WHY IT HAPPENS: COUNT++ IS *THREE* STEPS

💡 In plain terms: `count++` looks like one action, but the CPU does **three**: **read** the value, **add one**, **write** it back. The scheduler can pause a thread *between* any of them.

```
count++;           // looks atomic, it is NOT
// is really:
int tmp = count;    // 1. read
tmp = tmp + 1;      // 2. modify
count = tmp;        // 3. write ← another thread may have changed `count` in between
```

Any operation that **reads then writes** shared state has this hazard. The fix is *not* "avoid virtual threads". It's making that read-modify-write **indivisible**. That's tomorrow.

// TRY IT NOW (4 MIN): MAKE A RACE WITH YOUR OWN EYES

In `jshell`, let 1,000 virtual threads each add 1,000 to one **unguarded** counter:

```
jshell> class Counter { long value = 0; void inc() { value++; } }
jshell> var c = new Counter();
jshell> try (var ex = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
...>     for (int i = 0; i < 1_000; i++)
...>         ex.submit(() -> { for (int j = 0; j < 1_000; j++) c.inc(); return null; });
...> }
jshell> c.value           // expected 1_000_000 ...
```

`c.value` comes out **less than 1,000,000**, and a different number each run. Those are **lost updates**, live. You did nothing wrong; shared mutable state did. Tomorrow we make it right.

// THAT GAP IS THE WHOLE OF DAY 5

Today gave you the *power*: millions of cheap threads. Tomorrow gives you the *safety* to use it correctly.

💡 **Day 5: Concurrency II** answers "how do I share mutable state safely?": making operations **atomic**, guarding sections with **locks**, coordinating groups with **structured concurrency**, and carrying per-request context with **scoped values**.

The cheapest fix of all, though, you already know: **don't share mutable state**. Immutable records (Day 3) and one-task-per-thread sidestep most races entirely.

// COMMON TRAPS

- Calling `run()` instead of `start()`: runs inline on your thread; no concurrency at all.
- **Pooling virtual threads**: a fixed pool re-imposes the old ceiling. Use `newVirtualThreadPerTaskExecutor()` and submit one task each.
- **Virtual threads for CPU-bound work**: they don't beat your core count; use a sized pool.
- Believing old "`synchronized` pins" advice: true pre-JDK 24, fixed by JEP 491; in JDK 25 only native/FFM calls pin.
- Assuming `count++` is atomic: it's read-modify-write; two threads lose updates. (Day 5 fixes this properly.)

// RECAP, IN PLAIN TERMS

Today you learned how Java runs many things at once, and met the one hard problem:

- a **thread** is an OS thread with a private stack; the **scheduler** decides when it runs
- **concurrency** (juggling) $\neq$ **parallelism** (simultaneous); servers mostly juggle *waiting*
- prefer an `ExecutorService` to hand-managed `Thread` s: submit **tasks**, get `Future` s
- **virtual threads** (final in 25) make blocking cheap: one per task, no pools, no callbacks
- **shared mutable state** races: `count++` isn't atomic: the cliffhanger Day 5 resolves

// TODAY'S LAB

✍ **Feel both sides of concurrency.** Launch many virtual threads that each *block*, and prove the cost barely moves, then reproduce a **race condition** on a shared counter and watch updates vanish.

Brief & starter: `labs/day-04/` · single-file program, run with `java RaceLab.java` (no build).

Acceptance: cheap blocking demonstrated (all tasks finish), and a **reproducible lost-update race** printed (`actual < expected`). You'll *fix* it on Day 5. Today you only need to **make it fail**.

// TAKEAWAY

Modern Java lets you write **simple, blocking, one-task-per-thread** code that scales to millions. Virtual threads make the *power* cheap. But power isn't safety: the instant two threads touch the same mutable object, `count++` can lose an update, silently.

Tomorrow, Day 5: Concurrency II. We make shared state *safe*: atomics, locks, structured concurrency, and scoped values, so the cheap threads you just unlocked are also **correct**.